

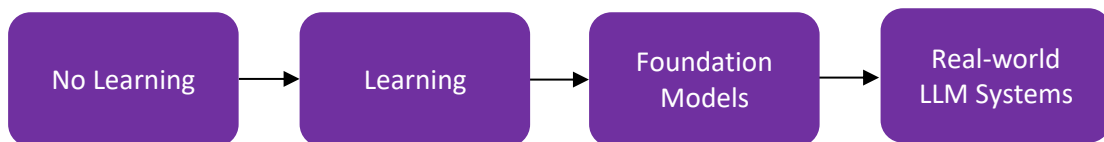
Learning and Capability Ladder

Diagram1 - Flow Diagram:

no learning → learning → foundation models → real-world LLM systems.

Diagram2 - Ladder Diagram:

Rule-based → ML → DL → LLMs → RL → GenAI → RAG (and the “no learning → learning → foundation models → real-world systems” story).

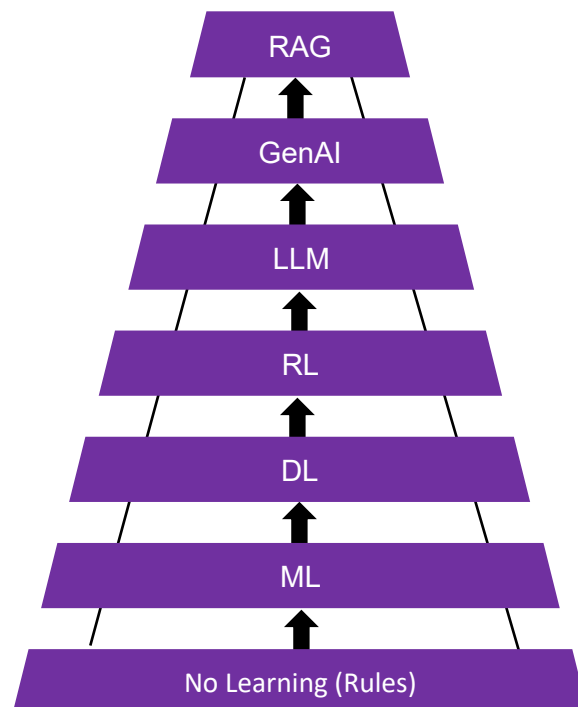


Ladder / Staircase Infographic (Most intuitive)

- A staircase with 7 steps, each step labelled.
- Bottom step: “No learning (Rules)”
- Middle: “Learning (ML, DL, RL)”
- Top: “Foundation models (LLMs, GenAI) → Systems (RAG)”

Icons Per Step:

Rules = checklist , ML = spreadsheet , DL = neural network , LLM = chat bubble , RL = joystick + trophy , GenAI = magic wand , RAG = magnifier + document



The Big Picture (Concept Map)

Think of it like layers:

- **Rule-based** = no learning (logic you write)
- **ML** = learns from **tables** (structured data)
- **DL** = ML using **deep neural nets** (great for unstructured data)
- **LLMs** = DL (Transformers) specialized for **language**
- **RL** = learning by **actions + rewards** over time (different training setup)
- **GenAI** = models that **generate** new content (text/images/audio/video)
- **RAG** = an **app pattern**: LLM + retrieval from your documents

Why We Teach in This Order

Simple → **Complex**, and each concept “unlocks” the next:

1. Start with **rules** (baseline: not everything is AI/ML)
2. Then **ML** (easiest learning: spreadsheet data)
3. Then **DL** (same idea, handles messy/unstructured inputs)
4. Then **LLMs** (a powerful DL subtype for language)
5. Then **RL** (different paradigm: decisions + rewards)
6. Then **GenAI** (generalise generation beyond text)
7. End with **RAG** (how companies actually build reliable LLM apps)

1) Rule-based systems

A rule-based system makes decisions using **human-written if/else rules**—it does **not** learn from data.

When to Use

- Logic is stable and clear
- You need predictable behaviour + easy explanation
- Quick baseline before ML

Example

Food delivery:

“If order value > ₹500 and first-time user → apply coupon.”

Strengths / Limits

- Fast, cheap, explainable
- Breaks when rules get too many; can’t adapt to changing patterns

Common Python Libraries

- Built-in Python (if, elif, dicts)
- pandas (apply rules on tables)
- Optional rule engines: durable_rules, business-rules, rule-engine

2) Machine Learning (ML)

ML learns patterns from **structured/tabular data** (rows and columns) to predict an outcome.

Typical Tasks

- Classification: churn yes/no, fraud yes/no
- Regression: predict delivery time
- Ranking: recommendations
- Anomaly detection: unusual transactions

Example

Churn prediction using: tenure, plan, usage, support tickets.

What Makes ML “Work”

You provide **features** (columns). The model learns relationships like:

- more complaints + low usage → churn risk increases

Strengths / Limits

- Great for business data; interpretable options exist
- Needs good features; struggles with raw images/audio/video

Common Python Libraries

- Core: scikit-learn
- Strong tabular performance: xgboost, lightgbm, catboost
- Data: pandas, numpy
- Tracking: mlflow

3) Deep Learning (DL)

DL is a subset of ML that uses **deep neural networks** to learn complex patterns—especially from **unstructured data**.

Why DL Matters

For images/audio, it's hard to define features manually. DL learns them:

- pixels → edges → shapes → objects

Example

Detect “road obstacle present?” from a camera frame.

Strengths / Limits

- Best for images/audio/text at scale
- Needs more data + compute; harder to interpret

Common Python Libraries

- Training: torch (PyTorch), tensorflow, keras
- Vision: torchvision, opencv-python, Pillow
- Audio: torchaudio, librosa

4) LLMs

LLMs are large Transformer-based deep learning models trained on massive text/code to **predict the next token**, enabling chat, summarisation, coding help, etc.

Why LLMs Feel Different

Many ML/DL models do **one job** (e.g., detect objects).

LLMs can do **many language tasks** with prompting.

Example

Customer support:

- “My order shows delivered but I didn’t receive it.”
- LLM can ask clarifying questions, follow policy, and draft a response.

Strengths / Limits

- General-purpose language ability
- Can hallucinate; must be grounded for factual/company info

Common Python Libraries

- Hosted LLMs: openai
- Open-source models: transformers (Hugging Face)
- Fast inference: vllm, llama-cpp-python
- App building: langchain, llama-index

5) Reinforcement Learning (RL)

RL is when an agent learns **what action to take** by trial-and-error to maximise reward over time.

Key pieces

- State = current situation
- Action = choice you make
- Reward = feedback score
- Policy = strategy

Example

Delivery routing strategy:

- reward: on-time deliveries (+), late deliveries (–)



Strengths / Limits

- Great for sequential decisions
- Needs a simulator or safe environment; can be complex/unstable

Common Python Libraries

- Environments: gymnasium
- Algorithms: stable-baselines3
- Scale: ray[rllib]
- Multi-agent: pettingzoo

6) Generative AI (GenAI)

GenAI models generate **new text/images/audio/video**, not just predict labels.

Important clarification

- **LLMs are GenAI for text**, but GenAI also includes image/audio/video generation.

Example

Marketing:

- “Generate a poster image + caption for 50% off pizza day.”

Strengths / limits

- Speeds up content creation and ideation
- Quality/safety needs guardrails; can generate incorrect content

Common Python Libraries

- Images: diffusers (Stable Diffusion pipelines), Pillow
- Text: same as LLMs (openai, transformers)
- Audio: torchaudio, plus model-specific libs (varies)

7) RAG

RAG (Retrieval-Augmented Generation) is a pattern where you **retrieve** relevant info from documents first, then give it to an LLM to answer accurately.

Why it Exists

LLMs may not know your private docs and may hallucinate.

RAG makes answers grounded in your data.

Example

HR bot:

- “How many casual leaves do we get?”
- RAG searches the HR policy PDF and uses that text to answer.



Typical RAG Steps

1. Split docs into chunks
2. Create embeddings for chunks
3. Store in a vector database
4. Retrieve top-k relevant chunks
5. Ask LLM with retrieved context

Common Python Libraries

- Frameworks: langchain, llama-index
- Vector search: faiss-cpu, chromadb, qdrant-client, weaviate-client
- Embeddings: sentence-transformers, or hosted embedding APIs
- PDF parsing: pypdf, pdfplumber

Quick “Which One Should I Use?”

Start from the input type and the goal:

- Clear logic → **Rule-based**
- Spreadsheet prediction → **ML**
- Images/audio/video understanding → **DL**
- Text generation/understanding → **LLM**
- Decisions over time with reward → **RL**
- Need to create content → **GenAI**
- Need answers from your documents → **RAG**

A Realistic “Product Pipeline” Example

Food delivery app:

1. **Rule-based:** Coupon eligibility & safety checks
2. **ML:** Predict late delivery risk (tabular signals: traffic, distance, prep time)
3. **DL:** Detect obstacles/traffic density from camera feeds
4. **LLM:** Support agent responses + summarising complaints
5. **RAG:** Support agent answers using company refund policy docs
6. **GenAI:** Marketing creatives (images + captions)
7. **RL (optional):** Optimise dispatch/routing decisions over time